

Page 1 (Title Page)

NETWORK TRAFFIC MONITORING AND
ANALYSIS PLATFORM

Project Report

Submitted to: [Madeeha Aman]

Course: Computer Networks

Semester: 4th Semester

Submitted by: [Muhammmad Sultan]

Roll number: [BCSF24M018]

Page 2

1. Introduction

Network traffic monitoring is a critical aspect of network administration and security. It involves capturing, analyzing, and interpreting data packets that travel across a network. This project aims to develop a real-time network traffic monitoring and analysis platform that demonstrates fundamental networking concepts.

The system captures live network packets, extracts relevant information such as source/destination IP addresses, protocol types, port numbers, and packet sizes. It then displays this information in a user-friendly web interface with filtering capabilities and real-time statistics.

2. Scope of Project

The scope of this project includes:

Area	Description
Packet Capture	Real-time capture of TCP, UDP, and ICMP packets from the live network interface
Data Extraction	Extract source IP, destination IP, protocol, packet size, port numbers
Service Mapping	Map destination ports to common services (HTTP, DNS, HTTPS, etc.)
Statistics	Calculate total packets, per-protocol counts, and average packet size

Area	Description
Filtering	Filter captured packets by protocol, source IP, and destination IP
Data Export	Export captured data to CSV format
Web Interface	Simple HTML/CSS interface with start/stop controls

Out of Scope:

- IPv6 packet support
- Packet payload inspection
- Real-time bandwidth graphs
- Remote monitoring across multiple devices

Page 3

3. Technology Stack

Category	Technology	Purpose
Backend Language	Python 3.13	Main programming language
Web Framework	Flask	REST API and web server
Packet Capture	Scapy	Live packet sniffing and parsing
Frontend	HTML5	Web interface structure
Styling	CSS3	Visual design and layout
Interactivity	JavaScript (Vanilla)	AJAX calls, dynamic updates, button handlers
Data Export	Python CSV module	Generate CSV files
Threading	Python threading	Background packet capture while serving web requests

4. Methodology

The project was developed using the following step-by-step methodology:

Step 1: Requirement Analysis

- Studied the project requirements from the assignment document
- Identified mandatory features: start/stop, filtering, statistics, service mapping
- Decided to implement real-time capture (bonus points) instead of CSV simulation

Step 2: Technology Selection

- Chose Python for backend due to Scapy library availability
- Selected Flask for lightweight web server
- Decided against complex frameworks to keep project simple

Step 3: System Architecture Design

- Designed modular structure with separate files for each responsibility
- Planned threading approach to run sniffer alongside web server
- Defined API endpoints for frontend-backend communication

Step 4: Backend Implementation

- Created `packet_processor.py` for port-to-service mapping
- Implemented `logger.py` for storing and filtering packet logs
- Built `statistics.py` for real-time metric calculation
- Developed `sniffer.py` with threading for non-blocking capture
- Integrated everything in `app.py` with Flask routes

Step 5: Frontend Implementation

- Designed HTML structure with statistics, filter, and log sections
- Added CSS for clean, readable interface
- Implemented JavaScript for AJAX polling and dynamic table updates

Step 6: Integration and Testing

- Connected frontend to backend API endpoints
- Tested with real network traffic (ping, web browsing)
- Debugged issues related to permissions and Scapy on Windows

Step 7: Feature Enhancements

- Added CSV export functionality
- Implemented filter reset button
- Added port-to-service mapping for 15+ common ports

Page 4

5. Challenges Faced and Solutions

Challenge	Solution
Scapy not capturing packets on Windows	Installed Npcap driver (WinPcap compatible mode). Restarted system.
"Template not found" error	Realized folder was named "template" instead of "templates" (Flask requirement). Renamed folder.
Packet sniffer blocking web server	Implemented Python threading to run sniffer in background thread while Flask runs in main thread.
High CPU usage from continuous polling	Reduced polling interval to 1 second and used efficient filtering.
Port mapping for unknown ports	Added fallback to show "Port-XXXX" format instead of failing.
Admin privileges required	Documented requirement clearly in README and report. Added warning message in app.py.
Filtering not working correctly	Modified filter logic in <code>logger.py</code> to use partial string matching for IP addresses.
CSV export showing empty data	Fixed by ensuring <code>packet_logger.get_all_logs()</code> returns the correct data structure.

6. Future Work

Given more time, the following improvements could be implemented:

1. **Interface Selection** - Allow users to choose which network interface to monitor (WiFi, Ethernet, Loopback)
2. **Packet Details View** - Click on a packet to view full headers and payload data
3. **Live Bandwidth Graph** - Visual representation of network usage over time using Chart.js
4. **PCAP Export/Import** - Save captures to standard PCAP format and load them for offline analysis
5. **Alert System** - Set thresholds for unusual traffic patterns (e.g., high ICMP rate = possible ping flood)
6. **Geolocation Mapping** - Map source/destination IP addresses to physical locations on a map
7. **Protocol Deep Inspection** - Parse application-layer protocols (HTTP requests, DNS queries)
8. **IPv6 Support** - Add support for IPv6 packet parsing
9. **Packet Capture Filters (BPF)** - Allow users to specify Berkeley Packet Filters (e.g., "port 80")

7. Conclusion

This project successfully demonstrates a fully functional real-time network traffic monitoring and analysis platform. All core requirements have been met including:

- Real-time packet capture (bonus achieved)
- Start/Stop monitoring functionality
- Protocol, source IP, and destination IP filtering
- Statistics display (total packets, per-protocol counts, average size)
- Port-to-service mapping
- Web-based interface with HTML/CSS
- CSV export functionality

The system provides an intuitive interface for users to monitor their network activity without requiring deep networking expertise. The modular code structure makes it easy to extend with additional features in the future.

Through this project, practical experience was gained in:

- Network protocol analysis using Scapy
- Multi-threaded programming in Python
- Web development with Flask
- Real-time data visualization
- Debugging system-level networking issues

8. References

Resource	URL/Description
Scapy Documentation	https://scapy.readthedocs.io/ - Packet manipulation library
Flask Documentation	https://flask.palletsprojects.com/ - Web framework
Npcap	https://npcap.com/ - Packet capture driver for Windows
W3Schools HTML/CSS	https://www.w3schools.com/ - Frontend reference
Python CSV Module	https://docs.python.org/3/library/csv.html - CSV export
Python Threading	https://docs.python.org/3/library/threading.html - Concurrency

Resource	URL/Description
IANA Port Numbers	https://www.iana.org/assignments/service-names-port-numbers/ - Official port mappings
